UNDERTALE-STYLE GAME MUSIC WITH MARKOV CHAINS: ORDER, MEMORIZATION AND ANTI-PLAGIARISM

Samuel Lucca Viana Miranda (2022043566)
Departamento de Ciência da Computação, UFMG

ABSTRACT

We present a symbolic generator of video-game music based on order-n Markov chains trained on 110 fan-made piano transcriptions of the UNDERTALE soundtrack. A state is a sonority (bar position, sounding pitches, articulation, duration), and a piece is a loop of 4-bar sections following a form such as AABA. Bach chorales serve only to validate the implementation of the Markov model; the proposed system targets game music. Without protection, the full-piano chain of order 1 copies 76% of its 6-event windows, whereas the melody-only chain of order 1 copies just 15% of its sections. A rejection-based anti-plagiarism constraint makes the order-1 piano chain usable in 20 of 20 seeds with 18% of windows still copied, yielding collages of fragments from different songs. Code, MIDI and audio are in the repository at https://github.com/Samuel-Wisart/Generative_Markov_Undertale_Music; the corpus is not distributed.

1. INTRODUCTION

Video-game soundtracks are short, loop-based and built on identifiable motives, which suits Markov models: the next state is sampled from a distribution over the last n states, counted from a corpus [1, 2]. The same properties make memorization a risk, since a high order reproduces long fragments of the corpus [3, 4], which matters for originality and copyright. We build a generator of *Undertale*-style music [5] (piano texture, 4/4, 4-bar sections, loop forms such as AABA) and quantify how order, texture and an anti-plagiarism constraint trade novelty against fidelity. Bach chorales are used only as a test bed to validate the Markov implementation (Section 4).

2. RELATED WORK

Markov composition dates back to the *Illiac Suite* [1] (survey: [2]). The Continuator uses variable-order models [3]; Markov constraints make generation steerable [6], and Papadopoulos et al. [4] avoid plagiarism by bounding verbatim copies, of which our max-copy rule is a simpler section-level variant. Game music has symbolic datasets such as NES-MDB [7] and VGMIDI, 823 piano arrangements of game soundtracks [8], close to our corpus. We study a deliberately simple, exactly measurable Markov baseline.

© Samuel Lucca Viana Miranda. Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution:** Samuel Lucca Viana Miranda, "Undertale-Style Game Music with Markov Chains: Order, Memorization and Anti-Plagiarism", in *Proc. of the 27th Int. Society for Music Information Retrieval Conf.,* Abu Dhabi, UAE, 2026.

3. METHOD

3.1 Corpus and Representation

The corpus has 110 fan-made MIDI piano transcriptions of the UNDERTALE soundtrack (not distributed). Discarding 10 files with changing meter and 3 almost empty ones leaves 97; the 80 with four-beat bars yield 1,115 sections of four bars. Notes are read with music21 [9] and quantized to 12 ticks per quarter note. Each piece is transposed, using the Krumhansl-Schmuckler key estimate [10], to the white-key collection of its relative C major / A minor, so major and minor pieces share one vocabulary.

A *state* is a slice of the score, $e = (b, \mathbf{p}, \mathbf{a}, d)$: bar position b, sorted sounding pitches $\mathbf{p}$ (empty for a rest), attack flags $\mathbf{a}$ and duration d. Slices start at each onset and at real rests (at least a third of a beat); smaller articulation gaps are absorbed by the previous note. We compare three textures: *melody* (highest note, G3 and above), *melody+bass* (highest and lowest notes) and full *piano*.

3.2 Markov Model

An order-n chain estimates, by counting over sections,

$$P(e_t \mid e_{t-n}, \ldots, e_{t-1}) = \frac{c(e_{t-n}, \ldots, e_t)}{c(e_{t-n}, \ldots, e_{t-1})}. \quad (1)$$

Each section is padded with n start symbols and closed by an end marker, so the chain learns when to stop. There is no smoothing; sampling uses $p \propto c^{1/T}$ with temperature T.

3.3 Generation and Constraints

A piece follows a form (AABA by default): each distinct letter is a section sampled from the chain, and repeated letters repeat it, as in game loops. A section is accepted only if (i) it ends with the end marker and lasts exactly four bars, (ii) it is not silent, and (iii) optionally, at most a fraction τ of its windows of k consecutive events occur verbatim in the corpus ($k = 8$ for melody and melody+bass, $k = 6$ for piano). MIDI is written with music21; a simple synthesizer (piano or chiptune timbre) renders audio for listening only.

Texture	n	Contexts	Coverage (%)	Copied windows (%)	Copied sections (%)	With $\tau = 0.3$: feasible / copied (%)	
Melody	1	2,450	73	8	15	20/20	6
	2	5,373	33	68	48	20/20	17
	3	6,337	21	91	69	20/20	27
	4	6,808	17	97	79	20/20	n/a
Melody+bass	1	8,304	28	46	32	20/20	11
	2	10,816	9	89	60	20/20	22
	3	11,556	8	96	68	0/20	n/a
	4	12,067	7	98	78	0/20	n/a
Piano	1	10,077	19	76	49	20/20	18
	2	12,036	6	98	77	0/20	n/a
	3	12,681	6	99	75	0/20	n/a
	4	13,128	5	100	78	0/20	n/a

Table 1. Effect of the order n on the Undertale corpus (60 pieces with form ABCD per row). *Coverage*: share of contexts of unseen pieces found in training (5-fold cross-validation by piece). *Copied*: windows ($k = 8$ for melody and melody+bass, $k = 6$ for piano) and whole sections found verbatim in the corpus. With $\tau = 0.3$: seeds (of 20) in which a piece could be generated, and copying left ("n/a": infeasible, or sections too short to contain a window).

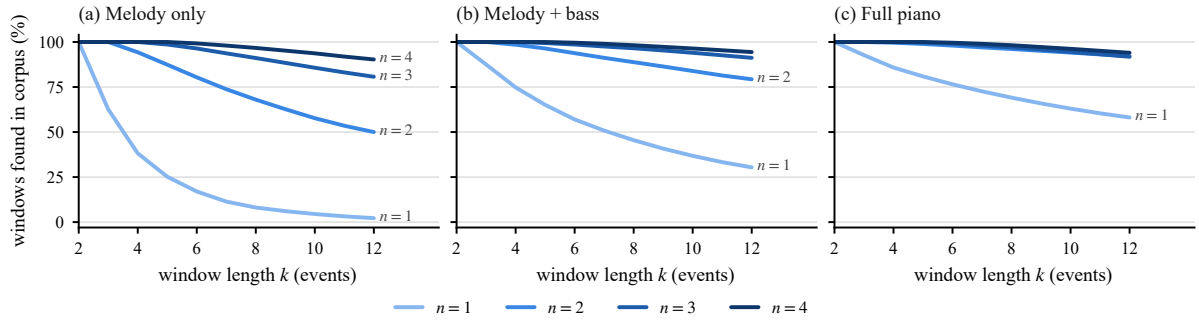


Figure 1. Novelty curves on the Undertale corpus: share of generated windows of length k found verbatim in the corpus. Every window up to $n+1$ events is a real corpus fragment by construction; where a curve leaves 100% marks the novelty horizon of that order.

4. EXPERIMENTS AND RESULTS

Validation on Bach chorales. The implementation was first checked on the 307 four-voice 4/4 chorales of music21 [9], whose behavior is well understood. The chain, sampling and rejection loop are the same code; only the corpus reader and phrase rules (fermatas, final tonic) are chorale-specific. The expected behavior was observed: pieces use only transitions present in the counts, phrases join metrically and end on the tonic, memorization grows with the order (whole-phrase copies from 1% to 86% for melody orders 1 to 5, and from 6% to 89% for four voices at orders 1 to 4), and parallel fifths and octaves stay between 0.17 and 0.24 per 100 transitions against 0.20 in the chorales. These pieces are not part of the proposed system.

Memorization (Table 1). Songs share little vocabulary: for an unseen piece only 19% of the order-1 piano contexts exist in training (60% for Bach), so the chain mostly recites the song it enters. Unprotected, the order-1 piano chain copies 76% of its windows and 49% of its sections; melody+bass copies 46% and 32%. Only the melody chain of order 1 is mostly original (8% and 15%), at the price of coherence: an order-1 chain only guarantees pairs of consecutive notes (Figure 1).

Anti-plagiarism. With $\tau = 0.3$ the order-1 piano chain works in 20 of 20 seeds with 18% of windows still copied, as do melody+bass orders 1 and 2 and melody orders 1 to 4; higher orders are infeasible (0 of 20) because the chain is almost deterministic. The defaults are melody $n = 2$, melody+bass $n = 1$ and piano $n = 1$, all with $\tau = 0.3$.

Examples. We rendered ten pieces. With $\tau = 0.3$, seven copy between 8 and 24% of their windows (these are in the repository); without it, order-1 piano seeds 1 to 3 copy 14, 100 and 100% (not distributed).

5. DISCUSSION AND FUTURE WORK

The chain has no global plan beyond the section loop. The corpus is small, made of fan transcriptions of uneven quality, and each song has its own vocabulary, which explains the strong memorization; "melody" is a heuristic (highest note from G3 up). The evaluation is objective, and copy rates are measured against the training data only, so they do not prove originality. A relative (interval-based) representation should recombine material across songs more freely; other extensions are a melody chain with a conditioned harmonization model [11], variable-order models with back-off [3], and listening tests.

6. AI USAGE STATEMENT

The software and the first draft of this manuscript were produced with the assistance of Claude Code (Anthropic, model Claude Sonnet 5.5) with help in the implementation and answering questions about the algorithms and getting references.

7. REFERENCES

[1] L. A. Hiller and L. M. Isaacson, *Experimental Music: Composition with an Electronic Computer*. New York, NY, USA: McGraw-Hill, 1959.

[2] C. Ames, "The Markov process as a compositional model: A survey and tutorial," *Leonardo*, vol. 22, no. 2, pp. 175–187, 1989.

[3] F. Pachet, "The Continuator: Musical interaction with style," *J. New Music Research*, vol. 32, no. 3, pp. 333–341, 2003.

[4] A. Papadopoulos, P. Roy, and F. Pachet, "Avoiding plagiarism in Markov sequence generation," in *Proc. 28th AAAI Conf. on Artificial Intelligence*, Québec City, Canada, 2014, pp. 2731–2737.

[5] T. Fox, "UNDERTALE soundtrack," Video game soundtrack, 2015.

[6] F. Pachet and P. Roy, "Markov constraints: Steerable generation of Markov sequences," *Constraints*, vol. 16, no. 2, pp. 148–172, 2011.

[7] C. Donahue, H. H. Mao, and J. McAuley, "The NES music database: A multi-instrumental dataset with expressive performance attributes," in *Proc. 19th Int. Society for Music Information Retrieval Conf.*, Paris, France, 2018, pp. 475–482.

[8] L. N. Ferreira and J. Whitehead, "Learning to generate music with sentiment," in *Proc. 20th Int. Society for Music Information Retrieval Conf.*, Delft, The Netherlands, 2019, pp. 384–390.

[9] M. S. Cuthbert and C. Ariza, "music21: A toolkit for computer-aided musicology and symbolic music data," in *Proc. 11th Int. Society for Music Information Retrieval Conf.*, Utrecht, The Netherlands, 2010, pp. 637–642.

[10] C. L. Krumhansl, *Cognitive Foundations of Musical Pitch*. New York, NY, USA: Oxford University Press, 1990.

[11] M. Allan and C. K. I. Williams, "Harmonising chorales by probabilistic inference," in *Advances in Neural Information Processing Systems 17*, 2005, pp. 25–32.